

AULA 14

Polimorfismo e Classe Abstrata

1. POLIMORFISMO

Na herança, podemos dizer que todo `LivroImpresso` é um `Livro`, pois `LivroImpresso` representa uma especialização da classe `Livro`. Isso significa que um `LivroImpresso` pode ser tratado como um `Livro` sem nenhum problema. Se alguém precisa comprar um `Livro` em nossa livraria, pode comprar um `LivroImpresso`, porque `LivroImpresso` é um `Livro`. Essa é a ideia da herança.



Para dar mais um passo em nossa aplicação, criaremos uma classe `SacolaDeCompras` para adicionar os livros adquiridos pelos clientes. Como vamos adicionar `LivrosImpressos` e também `AudioBooks`, uma maneira de resolver isso seria criando dois métodos `adicionar`, um para cada tipo de `Livro`, isto é, uma *sobrecarga*:

```
public class SacolaDeCompras {  
  
    public void adicionar(LivroImpresso livroImpresso) {  
        System.out.println(livroImpresso + " adicionado!");  
    }  
  
    public void adicionar(AudioBook audioBook) {  
        System.out.println(audioBook + " adicionado!");  
    }  
  
}
```

	SacolaDeCompras
	adicionar(livroImpresso): void
	adicionar(audioBook): void

Esse código funciona sem problema algum, mas está muito repetido e a cada novo tipo de `Livro` implementado, precisaremos criar um novo método `adicionar()` que receba esse novo tipo como parâmetro, o que seria trabalhoso e muito difícil de manter.

Para resolver isso, podemos dizer que tanto um `LivroImpresso` como um `AudioBook` são filhos da classe `Livro`. Podemos criar um único método `adicionar()`, que recebe um `Livro` (a *superclasse*) como parâmetro:

```
public class SacolaDeCompras {  
  
    public void adicionar(Livro item) {  
        System.out.println(item + " adicionado!");  
    }  
  
}
```

Podemos referenciar a objetos de `LivroImpresso` e `AudioBook` dessa forma mais genérica, pela sua classe-mãe `Livro`. Essa capacidade de um objeto poder ser referenciado de várias formas é conhecida como **Polimorfismo**.

Repare que conseguiremos passar um `AudioBook` para um método que recebe um `Livro` como argumento. Pense como uma sacola de compras com o seguinte aviso: “Permitida a inclusão apenas de Livros”. Um áudio book pode entrar nessa sacola? Sim, pois `AudioBook` é um `Livro`.



O polimorfismo não quer dizer que o objeto se transforma em outro, muito pelo contrário, um objeto nasce de um tipo e morre daquele tipo, o que pode mudar é a maneira como nos referimos a ele. O objeto passado em `adicionar(Livro item)` continua sendo um `LivroImpresso` ou `AudioBook`, apenas a forma como nos referimos a ele é que mudou. Mas, e se fizermos:

```
public void adicionar(Livro livro) {  
    System.out.println("Valor: " + livro.getTotal());  
}
```

Como o parâmetro recebido pelo método é um `Livro`, qual método `getTotal()` será executado: do `Livro`, `LivroImpresso` ou `AudioBook`? No Java, a invocação de método sempre vai ser decidida em tempo de execução: vai procurar o objeto na memória e, aí sim, decidir qual método deve ser chamado, sempre relacionando com sua classe de verdade, e não com a que estamos usando para referenciá-lo. Apesar de estarmos nos referenciando a um `LivroImpresso` como sendo um `Livro`, o método executado é o do `LivroImpresso`.

No dia em que criarmos uma classe `Kindle`, por exemplo, que é filha de `Livro`, precisaremos mudar a classe `SacolaDeCompras`? Não. Basta a classe `Kindle` reescrever os métodos que lhe parecerem necessários. É exatamente esse o poder do polimorfismo, juntamente com a reescrita de método: diminuir o acoplamento entre as classes, para evitar que novos códigos resultem em modificações em inúmeros lugares.

Repare que quem criou `SacolaDeCompras` pode nunca ter imaginado a criação da classe `Kindle` ou `LivroImpresso`. Contudo, não será necessário reimplementar esse controle em cada nova classe: reaproveitamos aquele código.

1.1. EXERCÍCIOS: POLIMORFISMO

- 1) Crie, no pacote `model`, a classe `SacolaDeCompras` que acumulará o valor total que está sendo comprado. Podemos fazer isso de uma forma bem simples, adicionando um atributo `total`:

```
public class SacolaDeCompras {  
    private double total;  
}
```

- 2) Inclua o método `adicionar()` que incrementa o total do `Livro` a cada adição:

```
public class SacolaDeCompras {  
    private double total;  
  
    public void adicionar(Livro livro) {  
        total += livro.getTotal();  
    }  
}
```

- 3) Crie o método `exibirTotal()`:

```
public void exibirTotal() {  
    System.out.println("-----");  
    System.out.println("Total: " + total);  
}
```

- 4) Abra a classe `TestaLivros` e, ao final do método `main`, instancie uma `SacolaDeCompras`:

```
static void main() {  
    ...  
    quartoLivro.setResumo("Ensina conceitos fundamentais sobre educação financeira.");  
  
    SacolaDeCompras sacola = new SacolaDeCompras();  
}
```

- 5) Adicione os quatro livros criados e exiba o total da sacola de compras:

```
SacolaDeCompras sacola = new SacolaDeCompras();  
sacola.adicionar(primeiroLivro);  
sacola.adicionar(segundoLivro);  
sacola.adicionar(terceiroLivro);  
sacola.adicionar(quartoLivro);  
sacola.exibirTotal();
```

- 6) Execute a classe para ver a saída.

- 7) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Criação de classe SacolaDeCompras"  
git push
```

2. CLASSE ABSTRATA

Agora, temos uma forma forte de representar os diferentes tipos de livro: além de `AudioBook`, também temos um `LivroImpresso`, que já provou ser um objeto diferente por ter comportamentos diferentes. Mas o que estamos vendendo quando fazemos `new` em um `Livro`?

```
Livro quintoLivro = new Livro();  
quintoLivro.setTitulo("O Objeto inválido");
```

O que, exatamente, vem a ser a nossa classe `Livro`, sendo que a nossa livraria vende apenas `LivroImpresso` e `AudioBook`? Para o nosso sistema, é inadmissível que um objeto seja apenas do tipo `Livro` (pode existir um sistema em que faça sentido ter objetos do tipo `Livro`, mas, no nosso caso, não).

Ao fazer uma venda, queremos saber ao certo o tipo de livro que está sendo comprado, nunca deveríamos permitir a venda de um `Livro`, mas sim de suas subclasses `LivroImpresso` ou `AudioBook`.

Então, o que é um `Livro` agora, um `LivroImpresso` ou um `AudioBook`? Na verdade, nenhum dos dois: um `Livro` é apenas uma abstração de tudo que os diferentes tipos de livro devem ter (herdar) em nossa livraria, ou seja, ela é uma classe que apenas idealiza um tipo, define apenas um rascunho e passa a servir exclusivamente para herança e polimorfismo, que são recursos bastante úteis e poderosos.

Para impedirmos que a classe `Livro` seja instanciada e utilizada, há uma forma bem simples: podemos simplesmente adicionar em sua declaração o modificador `abstract`. Veja:

```
public abstract class Livro {  
    ...  
}
```

A partir desse momento, não se pode mais criar um `Livro` e ponto final. O compilador vai impedir que qualquer código tente instanciar e, por ventura, utilizar um `Livro`. Apesar de não poder instanciar uma classe abstrata, você ainda pode (e muitas vezes deve) usá-la como referência, como no método adicionar da `SacolaDeCompras`.

Repare que quem criou `SacolaDeCompras` pode nunca ter imaginado a criação da classe `Kindle` ou `LivroImpresso`. Contudo, não será necessário reimplementar esse controle em cada nova classe: reaproveitamos aquele código.

2.1. EXERCÍCIOS: CLASSE ABSTRATA

- 1) Defina a classe `Livro` como `abstract`:

```
public abstract class Livro {  
}
```

- 2) Abra a classe `TestaReferencias` e observe a mensagem de erro “`'Livro' is abstract; cannot be instantiated`”. Troque o tipo para `LivroImpresso` ao invés de `Livro` para solucionar o problema.
- 3) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Definição de abstract em Livro"  
git push
```

3. MÉTODO ABSTRATO

Agora que `Livro` é uma classe abstrata, toda instância criada deve possuir um tipo específico e ser capaz de exibir suas informações particulares. Por exemplo, `LivroImpresso` deve apresentar a quantidade de páginas, enquanto `AudioBook` deve exibir informações como narrador, duração e versão.

Após a exibição dos dados comuns a todos os livros, cada tipo deve complementar a saída com seus detalhes específicos, conforme ilustrado abaixo:

```
public void exibir() {
    System.out.println("_____");
    System.out.println("Título: " + titulo);
    System.out.println("Autor: " + autor);
    System.out.println("Editora: " + editora);
    System.out.println("Disponibilidade: " + obterDisponibilidade());
    //Exibição dos detalhes específicos de cada tipo de livro
}
```

Uma possível solução é adicionar um método `exibirDetalhes()` à classe base e chamá-lo ao final do método `exibir()`.

```
public void exibir() {
    System.out.println("_____");
    System.out.println("Título: " + titulo);
    System.out.println("Autor: " + autor);
    System.out.println("Editora: " + editora);
    System.out.println("Disponibilidade: " + obterDisponibilidade());
    exibirDetalhes();
}

protected void exibirDetalhes() {
}
```

Ao executar esse código, apenas os dados básicos do livro (título, autor, editora etc.) serão exibidos. Como o método `exibirDetalhes()` não possui implementação, nenhum detalhe adicional da classe `Livro` será apresentado. Há uma forma simples de resolver o problema: poderíamos reescrever o método `exibirDetalhes()` na classe `LivroImpresso` e, dessa forma, a responsabilidade pela exibição das informações específicas fica delegada às subclasses, que poderão sobrescrever esse método conforme necessário.

```
public class LivroImpresso extends Livro {

    private int paginas;

    @Override
    protected void exibirDetalhes() {
        System.out.println("Páginas: " + paginas);
    }

}
```

Essa solução resolveria o problema, mas apresenta uma fragilidade importante: sempre que um novo tipo de livro fosse criado, seria necessário lembrar de ajustar a exibição padrão dos detalhes. Na prática, é pouco provável que todos os desenvolvedores se recordem desse requisito.

Se cada novo tipo de `Livro` possui uma forma específica de exibir seus detalhes, podemos adotar uma solução mais efetiva: como a classe `Livro` é abstrata, ela pode declarar métodos abstratos. Nesse caso, toda subclasse concreta (ou seja, não abstrata) será obrigada a implementar esses métodos, caso contrário, o código sequer será compilado.

Para tornar o método abstrato, na classe `Livro` basta adicionar o modificador `abstract` em sua declaração e remover todo o corpo, colocando apenas um ponto e vírgula. Observe:

```
protected abstract void exibirDetalhes();
```

Repare que não colocamos o corpo do método e usamos a palavra chave `abstract` para definir o mesmo. Por que não colocar corpo algum? Porque esse método nunca vai ser chamado, sempre que alguém chamar o método `exibirDetalhes()`, vai cair em uma das suas filhas, que realmente escreveram o método.



Qualquer classe que estender a classe `Livro` será obrigada a reescrever este método, tornando-o "concreto". Se não reescreverem esse método, um erro de compilação ocorrerá, como acontece na classe

AudioBook **agora:** Class 'AudioBook' must either be declared abstract or implement abstract method 'exibirDetalhes()' in 'Livro'.

Dessa forma, a responsabilidade pela exibição dos detalhes fica explícita em cada tipo de livro, eliminando a dependência da memória dos desenvolvedores e reduzindo a chance de erros durante a evolução do sistema.

Atenção: apenas classes abstratas podem ter métodos abstratos.

3.1. EXERCÍCIOS: MÉTODO ABSTRATO

- 1) Abre a classe `Livro` e crie o método `exibirDetalhes()` como abstrato.

```
public void exibir() {  
    System.out.println("_____");  
    System.out.println("Título: " + titulo);  
    ...  
}  
  
protected abstract void exibirDetalhes();
```

- 2) Faça a chamada para o método abstrato no final do método `exibir`.

```
public void exibir() {  
    System.out.println("_____");  
    System.out.println("Título: " + titulo);  
    ...  
    exibirDetalhes();  
}
```

- 3) Abra a classe `LivroImpresso` e, para corrigi-la, posicione o mouse sobre o erro, clique em [More actions...](#), escolha `Implement methods` e OK. Codifique a exibição do número de páginas:

```
@Override  
protected void exibirDetalhes() {  
    System.out.println("Páginas: " + paginas);  
}
```

- 4) Repita a operação com a classe `AudioBook`:

```
@Override  
protected void exibirDetalhes() {  
    System.out.println("Narrador: " + narrador);  
    System.out.println("Duração: " + duracao);  
    System.out.println("Versão: " + versao);  
}
```

- 5) Execute a classe `TestaLivros` para ver a saída.
- 6) Execute os comandos para envio das alterações ao *github*:

```
git status
```



```
git add .
```

```
git commit -m "Criação de método abstrato em Livro"
```

```
git push
```